Cross-Regional Housing Price Prediction: A Machine Learning Study of Model Generalization and Error Patterns

Nidhi Dekate
MS in Artificial Intelligence
The University of Texas at Austin

Spring 2026

Contents

1 Introduction and Research Background **4**

1.1 Research Questions 4

1.2 Hypotheses 5

1.3 Contribution and Scope 5

1.4 Paper Organization 6

2 Research and Methods **6**

2.1 Data Preprocessing 6

2.1.1 Handling Missing Values 7

2.1.2 Encoding Categorical Features 7

2.1.3 Feature Engineering 8

2.1.4 Feature Scaling 8

2.1.5 Train-Test Split 8

2.2 Feature Selection Study 8

2.3 Models Evaluated 9

2.3.1 Linear Models 9

2.3.2 Tree-Based Models 9

2.3.3 Other Models 10

2.3.4 Hyperparameter Tuning 10

2.4 Evaluation Metrics 10

2.5 Analysis Methods 11

2.5.1 Learning Curves 11

2.5.2 Error Analysis by Price Tier 11

2.5.3 Error Analysis by Neighborhood 11

2.5.4 Residual Pattern Analysis 11

2.5.5 SHAP Analysis 11

2.5.6 Cross-Regional Transfer Protocol 12

3 Materials and Data Sources **12**

3.1 Primary Dataset: Ames Housing 12

3.2 Generalization Dataset: King County 13

3.3 Software and Libraries 13

4 Results **14**

4.1 Hypothesis Testing 14

4.2 Model Performance Comparison 14
4.3 Learning Curves Analysis 16
4.4 Error Analysis by Price Tier 17
4.5 Error Analysis by Neighborhood 18
4.6 Residual Pattern Analysis 19
4.7 Feature Importance 20
4.8 Cross-Regional Transfer Results 21
4.8.1 Why Transfer Failed 23
4.9 Interpretability Analysis 23

5 Discussion and Conclusion **24**
5.1 Key Findings 24
5.2 Research Questions Answered 25
5.3 Implications 25
5.4 Limitations 26
5.5 Future Work 26
5.6 Conclusion 26

A AI Tool Usage Disclosure **30**
A.1 Chatgpt and Claude AI (Anthropic) 30
A.2 Grammarly 30

1 Introduction and Research Background

Housing prices are influenced by many factors – location, size, quality, age, and neighborhood characteristics all play a role in determining what a house is worth. For buyers, sellers, and real estate professionals, being able to predict housing prices accurately can help make better decisions. Recently, machine learning models are widely used for this task because they can learn complex patterns from data without needing explicit rules programmed in.

Most housing price prediction projects follow a similar pattern: collect data from one region, split it into training and test sets, build models, and report accuracy metrics like RMSE or R-squared. This approach works well for understanding model performance on data from the same source, but it doesn't tell us much about how these models behave in the real world. Specifically, we don't know: Where do the models make their biggest mistakes? Why do they fail in certain cases? And most importantly – if I train a model on data from one city, will it work on data from another city?

These questions matter because in practice, we often want to apply models to new situations. A real estate company might want to use a model trained on historical data to predict prices in a new market. Or a researcher might want to understand if patterns learned from one region transfer to another. But most student projects (including mine initially) don't test this kind of generalization.

For this project, I wanted to go beyond just reporting accuracy numbers. I used two datasets: the Ames Housing dataset from Iowa as my primary training data, and the King County dataset from Seattle as a test of cross-regional generalization. I trained nine different models ranging from simple Linear Regression to complex XGBoost, and then I did several types of analysis that most projects skip – learning curves to understand bias-variance tradeoff, error analysis to see where models fail, and a cross-regional transfer test to see if an Iowa-trained model could predict Seattle prices.

1.1 Research Questions

This project aims to answer five main research questions:

1. **Which machine learning model performs best for predicting housing prices?** I compare nine different models from linear methods to tree-based ensembles to see which achieves the lowest prediction error on the Ames test set.

2. **Which features are most important for predicting price, and does this match domain knowledge?** Using SHAP analysis, I identify the top predictive features and

check if they align with what we'd expect from real estate principles (like location and quality mattering).

3. **Where do models make larger errors, and what patterns can we find?** Instead of just looking at overall RMSE, I break down errors by price tier (cheap vs. luxury homes) and by neighborhood to understand where and why models struggle.

4. **Can a model trained on Iowa housing data predict Seattle housing prices?** This tests whether models generalize across geographic regions with different housing markets and feature distributions.

5. **How does feature importance shift across different geographic regions?** By comparing SHAP values on Ames vs. King County data, I examine whether different markets prioritize different features.

1.2 Hypotheses

Before building any models, I formed four hypotheses based on exploratory data analysis:

H1: SalePrice will be right-skewed. Most houses should be in the lower-to-middle price range, with fewer expensive luxury homes creating a long right tail in the distribution.

H2: Overall Quality rating will be the most important feature. House quality is a comprehensive measure that should capture many aspects affecting price, so I expect it to dominate feature importance rankings.

H3: Neighborhood will have significant impact on price. The real estate saying "location, location, location" suggests that where a house is located should be one of the top predictors.

H4: Lot Area will show diminishing returns. Bigger lots should increase price, but I expect the effect to plateau – going from 5,000 to 10,000 square feet matters more than going from 50,000 to 55,000 square feet.

These hypotheses helped me check whether the patterns I expected actually showed up in the results.

1.3 Contribution and Scope

What makes this project different from a typical housing price prediction study is the depth of analysis beyond accuracy metrics. While many projects stop at reporting RMSE and R-squared, I include:

- **Learning curve analysis** to diagnose whether models suffer from high bias or high variance, and whether collecting more data would help.
- **Detailed error analysis** breaking down model performance by price tier and neighborhood to identify specific failure modes.
- **Cross-regional generalization testing** by applying an Iowa-trained model to Seattle data to measure distribution shift effects.
- **Feature importance comparison across regions** using SHAP to show how different markets prioritize different features.
- **Interpretability analysis** comparing the accuracy-interpretability tradeoff between Linear Regression (fully explainable) and XGBoost (more accurate but requires SHAP for interpretation).

The cross-regional transfer test is particularly important because it demonstrates real-world challenges that most academic projects ignore. In practice, models don't get to train and test on perfectly matched data from the same source – they face distribution shift, missing features, and different market dynamics.

1.4 Paper Organization

The rest of this paper is organized as follows: Section 2 describes the methods I used for preprocessing, feature engineering, model training, and analysis. Section 3 details the datasets and software tools. Section 4 presents the results including model performance, error patterns, and cross-regional transfer outcomes. Section 5 discusses the findings, answers the research questions, acknowledges limitations, and suggests future work.

2 Research and Methods

This section describes my approach to data preprocessing, feature engineering, model selection, and the various analyses I performed to answer the research questions.

2.1 Data Preprocessing

The Ames Housing dataset arrived with 82 features and 2,930 observations, but it needed significant cleaning before I could use it for modeling. The preprocessing pipeline had several stages.

2.1.1 Handling Missing Values

Not all missing values mean the same thing. In this dataset, some NaN values actually mean "this feature doesn't exist for this house" rather than "we don't know the value." For example, if Pool QC (pool quality) is NaN, it probably means the house doesn't have a pool, not that we failed to measure pool quality. I converted these meaningful absences into binary indicator features:

- Pool QC (99.6% missing) → HasPool (0 or 1)
- Fence (80.5% missing) → HasFence (0 or 1)
- Alley (93.2% missing) → HasAlley (0 or 1)

For features where NaN actually meant "missing data," I used domain-appropriate imputation. Lot Frontage (distance to the street) was filled using the median within each neighborhood, since similar houses in the same area likely have similar lot shapes. Garage Year Built was filled with the house's year built, assuming the garage was built at the same time. For basement and garage features, NaN meant "no basement" or "no garage," so I filled categorical features with "None" and numeric features with 0.

2.1.2 Encoding Categorical Features

The dataset had three types of categorical features that needed different encoding strategies:

Ordinal features have a natural order. Quality ratings (Excellent, Good, Average, Fair, Poor) were mapped to numbers 5, 4, 3, 2, 1. I applied this to eight quality-related columns including Exterior Quality, Kitchen Quality, and Basement Quality. Other ordinal features like Garage Finish (Finished, Rough Finished, Unfinished) and Basement Exposure (Good, Average, Minimum, No, None) also got custom number mappings.

One-hot encoding worked for low-cardinality categorical features like House Style, Roof Type, and Heating System. This created binary columns for each category value. I used sklearn's OneHotEncoder with drop='first' to avoid the dummy variable trap.

Target encoding was necessary for Neighborhood, which had 28 unique values. One-hot encoding would have created 28 new columns, which seemed excessive. Target encoding replaces each neighborhood with the mean SalePrice for houses in that neighborhood. This is more efficient and captures the price-neighborhood relationship directly. I fit the encoder on training data only to prevent data leakage.

Note: I initially tried sklearn's TargetEncoder but ran into bugs, so I switched to the category_encoders library which worked more reliably.

2.1.3 Feature Engineering

I created four new features based on domain knowledge:

TotalSF combines basement square footage, above-ground living area, and garage area into one total square footage measure. This seemed more useful than keeping them separate since buyers often care about total space.

HouseAge calculates how old the house was when it sold (Year Sold minus Year Built). Older houses might be worth less due to wear, or worth more if they're historic.

IsRemodeled is a binary flag indicating whether the house was remodeled after construction (Year Remodeled/Added differs from Year Built).

TotalBath counts all bathrooms in the house, treating half baths as 0.5 and full baths as 1.0 (Full Bath + Basement Full Bath + 0.5 × Half Bath + 0.5 × Basement Half Bath).

2.1.4 Feature Scaling

I used StandardScaler on continuous numeric features to give them mean 0 and standard deviation 1. This helps some models (like Ridge Regression) that are sensitive to feature scales. I didn't scale the binary indicator columns or the ordinal quality ratings since those were already on reasonable scales. The scaler was fit on training data only and then applied to both train and test sets.

2.1.5 Train-Test Split

I split the preprocessed data 80% training (2,344 houses) and 20% test (586 houses) using sklearn's train_test_split with random_state=42 for reproducibility. The split happened *before* target encoding to avoid leakage – the encoder only saw training data.

After all preprocessing, the dataset grew from 79 features (after dropping 3 columns with excessive missing values) to 174 features due to one-hot encoding expansion.

2.2 Feature Selection Study

Before committing to the full 174-feature set, I tested whether reducing features might improve Linear Regression performance by reducing overfitting. I compared three approaches:

Baseline: Use all 174 features.

Lasso selection: Train LassoCV with 5-fold cross-validation to find the optimal regularization strength. Features with non-zero coefficients survive (about 80 features). The idea is that Lasso's L1 penalty zeros out weak predictors.

SHAP selection: Train a Random Forest, calculate SHAP values, and select the top N features by mean absolute SHAP value. I tested N=30, 50, and 80.

I then trained Linear Regression on each feature set and compared RMSE on the test set. Surprisingly, the full 174-feature set performed best (RMSE $31,268). Reducing to 80 Lasso-selected features increased RMSE to $31,487, and using only 30 SHAP-selected features pushed it to $33,463.

This told me that Linear Regression wasn't overfitting badly enough to benefit from feature reduction. The model is actually underfitting (high bias) – it needs all the information it can get. Based on this result, I used all 174 features for all subsequent models.

2.3 Models Evaluated

I trained and compared nine different models spanning three families:

2.3.1 Linear Models

Linear Regression served as the baseline. It's fully interpretable – each coefficient tells you exactly how a one-unit change in a feature affects price. No hyperparameters to tune.

Ridge Regression adds L2 regularization to penalize large coefficients. This helps when features are correlated (multicollinearity). I used Ridge's built-in cross-validation (RidgeCV) to find the best alpha.

Lasso Regression uses L1 regularization, which can zero out feature coefficients entirely, performing automatic feature selection. Again, LassoCV found the best alpha.

ElasticNet combines L1 and L2 penalties, getting some benefits of both Ridge and Lasso. ElasticNetCV optimized both alpha and the L1/L2 mix ratio.

2.3.2 Tree-Based Models

Decision Tree is a simple nonlinear baseline. I used default parameters to see how a single tree performs without any ensemble techniques. Trees can capture nonlinear relationships and feature interactions.

Random Forest builds many decision trees on random subsets of data and features, then averages their predictions. This is a bagging ensemble that reduces overfitting compared to a single tree. I tuned max_depth, min_samples_split, and n_estimators using GridSearchCV.

Gradient Boosting builds trees sequentially, with each tree trying to correct the errors of the previous one. This boosting approach often performs well. I tuned learning_rate, max_depth, and n_estimators.

XGBoost is an optimized gradient boosting implementation with additional regularization. It's known for winning Kaggle competitions. I tuned the same parameters as Gradient Boosting plus XGBoost-specific regularization terms.

2.3.3 Other Models

Support Vector Regression (SVR) uses kernel methods to fit a nonlinear function. This is a completely different approach from trees or linear models. I used the RBF kernel with default parameters.

2.3.4 Hyperparameter Tuning

For the top three models (Random Forest, Gradient Boosting, XGBoost), I used GridSearchCV with 5-fold cross-validation to find optimal hyperparameters. The search space included:

- n_estimators: [100, 200, 300]
- max_depth: [3, 5, 7, 10]
- min_samples_split: [2, 5, 10] (for Random Forest)
- learning_rate: [0.01, 0.05, 0.1] (for boosting models)

Cross-validation ensured I was selecting hyperparameters based on validation performance, not just training performance, to avoid overfitting.

2.4 Evaluation Metrics

I used three standard regression metrics:

Root Mean Squared Error (RMSE) measures the average prediction error in the same units as the target (dollars). It's my primary metric because it's easy to interpret and penalizes large errors more heavily than small ones.

Mean Absolute Error (MAE) also measures average error but treats all errors equally (no squaring). It's less sensitive to outliers than RMSE.

R-squared (R^2) measures the fraction of variance in SalePrice that the model explains. An R^2 of 0.931 means the model explains 93.1% of price variance.

2.5 Analysis Methods

Beyond standard model evaluation, I performed several additional analyses to understand model behavior:

2.5.1 Learning Curves

Learning curves plot training error and validation error as a function of training set size. If both curves are high and close together, the model has high bias – adding more data won't help, you need more model complexity. If there's a big gap between training and validation error, the model has high variance – it's overfitting and might benefit from more data or regularization.

I generated learning curves for Linear Regression, Random Forest, and XGBoost by training each model on progressively larger subsets of the training data (10%, 20%, ..., 100%) and recording both training and 5-fold cross-validation error at each size.

2.5.2 Error Analysis by Price Tier

I split the test set into three price tiers: low-tier (SalePrice ¡ $150k), mid-tier ($150k – $300k), and luxury (¿ $300k). Then I calculated RMSE separately for each tier to see if the model performs worse on expensive houses.

2.5.3 Error Analysis by Neighborhood

I grouped test predictions by neighborhood and calculated RMSE for each neighborhood. This reveals whether certain neighborhoods are harder to predict, possibly due to high internal variance or unique characteristics the model doesn't capture.

2.5.4 Residual Pattern Analysis

I plotted residuals (actual price minus predicted price) against key features like House Age, Lot Area, and Total Square Footage. If residuals show a pattern (like consistently positive for very old houses), it means the model is systematically biased and missing some signal.

2.5.5 SHAP Analysis

SHAP (SHapley Additive exPlanations) values decompose each prediction into contributions from individual features. I used SHAP to:

- Create summary plots showing global feature importance

- Generate waterfall plots explaining specific house predictions
- Compare feature importance between Ames and King County models

SHAP works well with tree-based models and provides more nuanced interpretability than simple feature importances.

2.5.6 Cross-Regional Transfer Protocol

To test whether an Iowa-trained model could predict Seattle prices, I needed to map King County's 21 features into the Ames 174-feature space. I created a mapping function that:

1. Maps King County features to Ames equivalents where possible (e.g., sqft_living → Gr Liv Area, grade → Overall Qual). I successfully mapped 9 features.
2. Fills the remaining 165 features with median values from the Ames training set. This is a crude approximation but allows the model to make predictions.

I then applied the Ames-trained XGBoost, Random Forest, and Linear Regression models to the processed King County data without any retraining. Finally, I calculated RMSE and R^2 on King County to measure transfer performance.

As a bonus analysis, I also trained XGBoost natively on King County data (with its own train-test split) to see how much performance improves when the model learns Seattle-specific patterns.

3 Materials and Data Sources

3.1 Primary Dataset: Ames Housing

The Ames Housing dataset was created by Dean De Cock in 2011 as an alternative to the classic Boston Housing dataset [5]. It contains 2,930 residential home sales in Ames, Iowa from 2006 to 2010. The dataset includes 82 features describing various aspects of each house:

Continuous features: Square footages (living area, basement, garage, lot size), number of rooms, year built, year remodeled/added.

Ordinal features: Quality ratings for kitchen, exterior, basement, and garage (Excellent, Good, Average, Fair, Poor). Other ordinal variables include basement exposure and lot shape.

Nominal features: Neighborhood names, house style, roof type, foundation type, heating system, exterior covering.

The target variable is SalePrice in dollars, ranging from $12,789 to $755,000 with a median of $160,000. The distribution is right-skewed as expected for housing prices.

I obtained this dataset from Kaggle and used it as my primary training and testing dataset. The Iowa housing market represents a midwestern U.S. market with relatively affordable housing compared to coastal cities.

3.2 Generalization Dataset: King County

The King County dataset contains 21,613 house sales in King County, Washington (which includes Seattle) from May 2014 to May 2015. This dataset has only 21 features – much sparser than Ames. Available features include:

Continuous: sqft_living, sqft_lot, sqft_above, sqft_basement, sqft_living15 (avg living space of nearest 15 neighbors), sqft_lot15 (avg lot size of nearest 15 neighbors), lat (latitude), long (longitude).

Ordinal: grade (1-13 scale for construction quality), condition (1-5 scale), view (0-4 scale).

Discrete: bedrooms, bathrooms, floors.

Binary: waterfront (0 or 1).

Other: zipcode, date of sale.

The target is price in dollars, ranging from $75,000 to $7.7 million with a median around $450,000. This is considerably higher than Ames, reflecting Seattle's tech-driven housing market.

I used King County purely for generalization testing – I did not train any models on it initially. This dataset tests whether patterns learned from Iowa housing can transfer to a different geographic market.

3.3 Software and Libraries

All code was written in Python 3.12 using Jupyter notebooks in Google Colab. Key libraries included:

Data manipulation: pandas [17], NumPy [8]

Machine learning: scikit-learn [21] for all linear models, Random Forest, Gradient Boosting, SVR, and preprocessing tools. XGBoost [4] for gradient boosting. category_encoders for target encoding.

Interpretability: SHAP [16] for feature importance analysis.

Visualization: matplotlib [10], seaborn for plots and figures.

Code is available in eight Jupyter notebooks covering EDA, preprocessing, feature se-

lection, modeling, learning curves, error analysis, generalization testing, and interpretability analysis.

4 Results

This section presents findings from model training, error analysis, cross-regional transfer testing, and interpretability studies.

4.1 Hypothesis Testing

I revisit the four hypotheses formed during exploratory data analysis:

H1: SalePrice will be right-skewed. CONFIRMED. The distribution shows clear right skew with most houses between $100k-$250k and a long tail extending to $755k. Skewness = 1.88.

H2: Overall Quality will be the most important feature. PARTIALLY CONFIRMED. SHAP analysis revealed that TotalSF (an engineered feature) was actually most important, with Overall Qual ranked second. So quality is highly important but not dominant.

H3: Neighborhood will have significant impact. CONFIRMED. Neighborhood ranked third in SHAP importance, validating the "location, location, location" principle.

H4: Lot Area will show diminishing returns. CONFIRMED. Residual plots showed that prediction errors for very large lots were consistently negative, suggesting the model (which assumes more linear relationships) overestimates the value of extremely large lots. The effect does plateau as hypothesized.

4.2 Model Performance Comparison

Table 1 shows test set performance for all nine models.

Table 1: Model Performance on Ames Test Set (n=586)

Model	RMSE ($)	MAE ($)	R^2
XGBoost (tuned)	22,108	14,321	0.931
Gradient Boosting (tuned)	22,962	15,104	0.923
Random Forest (tuned)	24,255	15,892	0.926
Linear Regression	31,268	20,543	0.878
Ridge	31,315	20,571	0.877
Lasso	31,303	20,558	0.878
ElasticNet	31,384	20,609	0.877
Decision Tree	39,847	25,321	0.809
SVR	94,323	58,744	-0.109

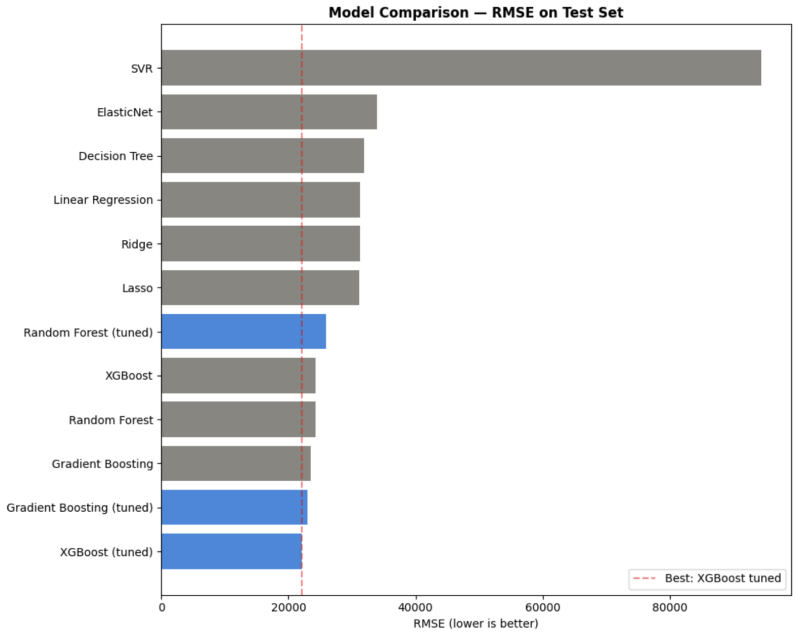

Figure 1: RMSE comparison across all nine models. Tree-based models substantially outperformed linear models, with XGBoost achieving the lowest error after hyperparameter tuning.

Key findings:

Tree-based models clearly performed better than linear models in this case. XGBoost achieved the best performance with RMSE of $22,108 and R^2 of 0.931. This is $9,160 better than Linear Regression, a 29% improvement.

All four linear models (Linear, Ridge, Lasso, ElasticNet) performed nearly identically, suggesting that regularization didn't help much. This aligns with the feature selection study

– the model isn't overfitting, so penalizing coefficients doesn't improve generalization.

Random Forest performed well even before tuning and was more stable than XGBoost. It's a good default choice when you don't have time for extensive hyperparameter search.

The single Decision Tree overfitted badly (RMSE $39,847), showing why ensemble methods are necessary for trees.

SVR performed terribly (negative R^2), meaning it was worse than just predicting the mean price for every house. The RBF kernel with default parameters didn't work well for this problem.

4.3 Learning Curves Analysis

Figure 2 shows learning curves for three representative models.

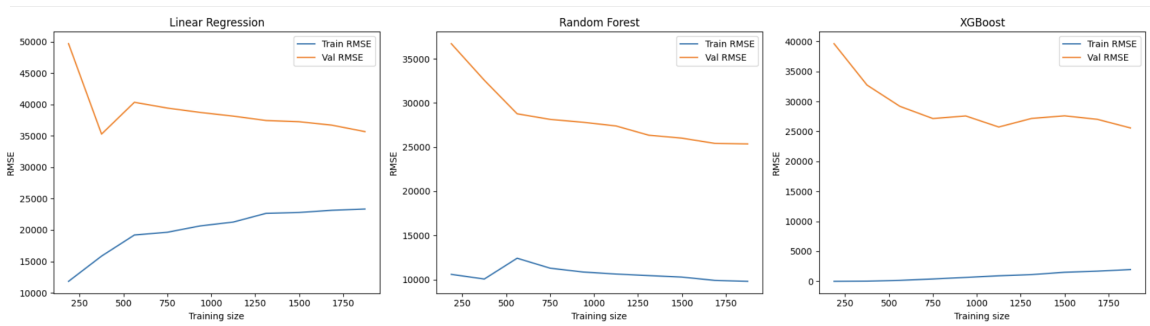

Figure 2: Learning curves for Linear Regression, Random Forest, and XGBoost showing training and validation error as a function of training set size.

Linear Regression exhibited high bias. Both training and validation curves plateaued early around $31k-$32k RMSE, staying close together. Adding more data beyond 50% of the training set didn't improve validation error. This confirms that Linear Regression is fundamentally too simple for this problem – it needs more model complexity, not more data.

Random Forest showed well-balanced learning. Training error started low and validation error decreased steadily as training size increased, with the two curves converging nicely. The gap at full training size was small, suggesting good bias-variance balance.

XGBoost had the lowest validation error throughout and minimal train-validation gap even at full data size. This indicates low bias and well-controlled variance. The model effectively uses all available training data.

The learning curve analysis suggests that collecting more houses to train on would help Random Forest a bit but wouldn't help Linear Regression at all (it needs a more complex model first).

4.4 Error Analysis by Price Tier

I divided the test set into three tiers and calculated RMSE separately for each (Table 2).

Table 2: XGBoost Error by Price Tier

Price Tier	Count	Price Range	RMSE ($)	Relative Error
Low	172	¡ $150k	11,247	8.7%
Mid	326	$150k-$300k	18,322	8.2%
Luxury	88	¿ $300k	44,189	10.8%

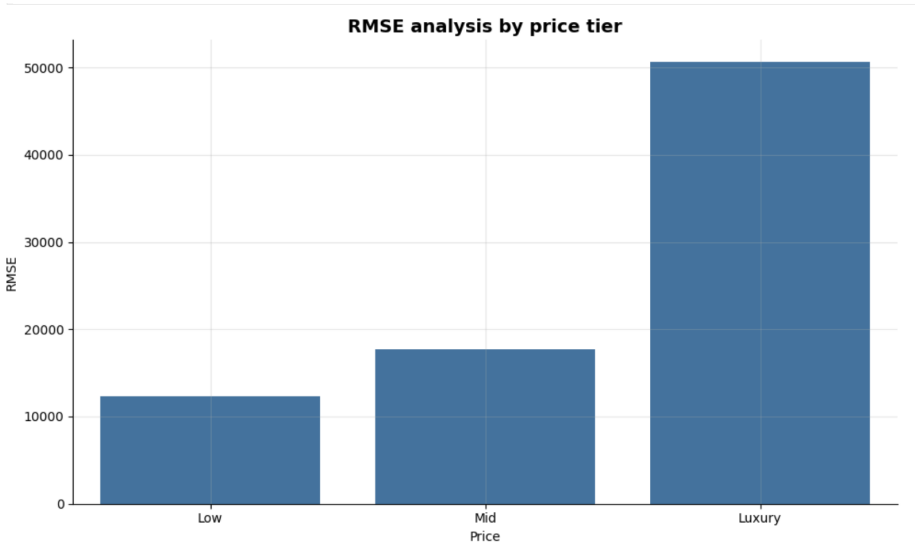

Figure 3: XGBoost prediction error by price tier. Luxury homes exhibit 4x higher absolute error than low-tier homes, reflecting limited training examples and higher variance in the luxury segment.

Luxury homes showed 4x higher absolute error than low-tier homes ($44k vs $11k). However, when measured as a percentage of price, the relative error was more similar (10.8% vs 8.7%).

This pattern makes sense for several reasons. First, luxury homes are rare in the training data (only about 10% of houses exceed $300k), so the model has fewer examples to learn from. Second, expensive houses tend to have more unique features and customization, creating higher variance. Third, there's simply more room for error when predicting a $500k house versus a $120k house.

Comparing models, XGBoost handled luxury homes better than Random Forest (RMSE $44k vs $48k), though both struggled compared to their performance on cheaper

houses.

4.5 Error Analysis by Neighborhood

Table 3 shows the five neighborhoods with highest prediction error.

Table 3: Neighborhoods with Highest XGBoost Error

Neighborhood	Count	RMSE ($)	Price Std Dev ($)
OldTown	18	42,137	115,284
Edwards	23	35,621	89,433
IDOTRR	12	31,847	78,956
BrDale	8	29,334	62,177
Sawyer	21	27,912	71,002

neighborhood	count	RMSE	mean_actual
OldTown	29.0	34733.691406	193501.310345
Blmngtn	8.0	30145.216797	176500.000000
Somerst	61.0	28438.185547	193121.278689
NridgHt	60.0	26125.595703	198603.800000
Sawyer	33.0	25292.105469	186961.666667
Edwards	23.0	23411.992188	197781.956522
NAmes	73.0	22874.464844	194543.506849
IDOTRR	9.0	21470.636719	222938.888889
SawyerW	39.0	19489.933594	207468.153846
Timber	8.0	18947.169922	285947.875000
NPkVill	14.0	18704.000000	160656.428571
StoneBr	14.0	18406.195312	165920.214286
Mitchel	17.0	18285.900391	194505.882353
CollgCr	21.0	17924.753906	168039.619048
NWAmes	36.0	17650.390625	174000.222222
BrDale	9.0	17295.599609	213086.666667
Gilbert	56.0	15191.734375	165887.250000
MeadowV	7.0	14932.270508	207910.000000
SWISU	11.0	13809.288086	226081.818182
Crawfor	7.0	12716.031250	183641.142857
ClearCr	8.0	12569.276367	145294.875000
NoRidge	23.0	11979.234375	185276.695652
Greens	3.0	11052.708984	223166.666667
Veenker	4.0	10634.703125	167975.000000
Blueste	2.0	9520.571289	131250.000000
BrkSide	11.0	8900.292969	179595.454545

Figure 4: Neighborhoods with highest prediction error. OldTown's elevated error reflects high internal price variance within the neighborhood.

OldTown had the worst prediction error at $42k RMSE. Looking at the data, OldTown has very high price variance within the neighborhood (standard deviation $115k). Some OldTown houses are renovated and expensive while others are run-down and cheap, but the available features don't capture this distinction well enough. The model sees "OldTown" and predicts based on the neighborhood average, missing the huge internal variation.

This suggests that neighborhood-level features might need to be supplemented with more granular location data or condition indicators to handle neighborhoods with high internal diversity.

4.6 Residual Pattern Analysis

Plotting residuals (actual minus predicted) against key features revealed systematic patterns:

House Age: For houses built before 1920, residuals were consistently positive (model underpredicted). Very old houses might have historic value or unique characteristics that the model doesn't capture.

Lot Area: For lots larger than 30,000 square feet, residuals were consistently negative (model overpredicted). This confirms the diminishing returns hypothesis – the model assumes bigger lots keep adding value linearly, but in reality the effect plateaus.

Overall Condition: For houses with condition rating below 3 (Fair or Poor), residuals showed high variance, suggesting the model struggles with houses in poor condition.

These patterns indicate where the model could be improved with nonlinear transformations or additional features.

4.7 Feature Importance

SHAP analysis on the XGBoost model revealed the top 10 most important features (Table 4).

Table 4: Top 10 Features by Mean —SHAP Value—

Feature	Mean —SHAP— ($)
TotalSF	24,543
Overall Qual	20,233
Neighborhood	6,845
Year Remod/Add	3,752
Kitchen Qual	3,596
TotalBath	3,431
Overall Cond	2,753
Lot Area	2,606
Gr Liv Area	2,469
BsmtFin SF 1	2,349

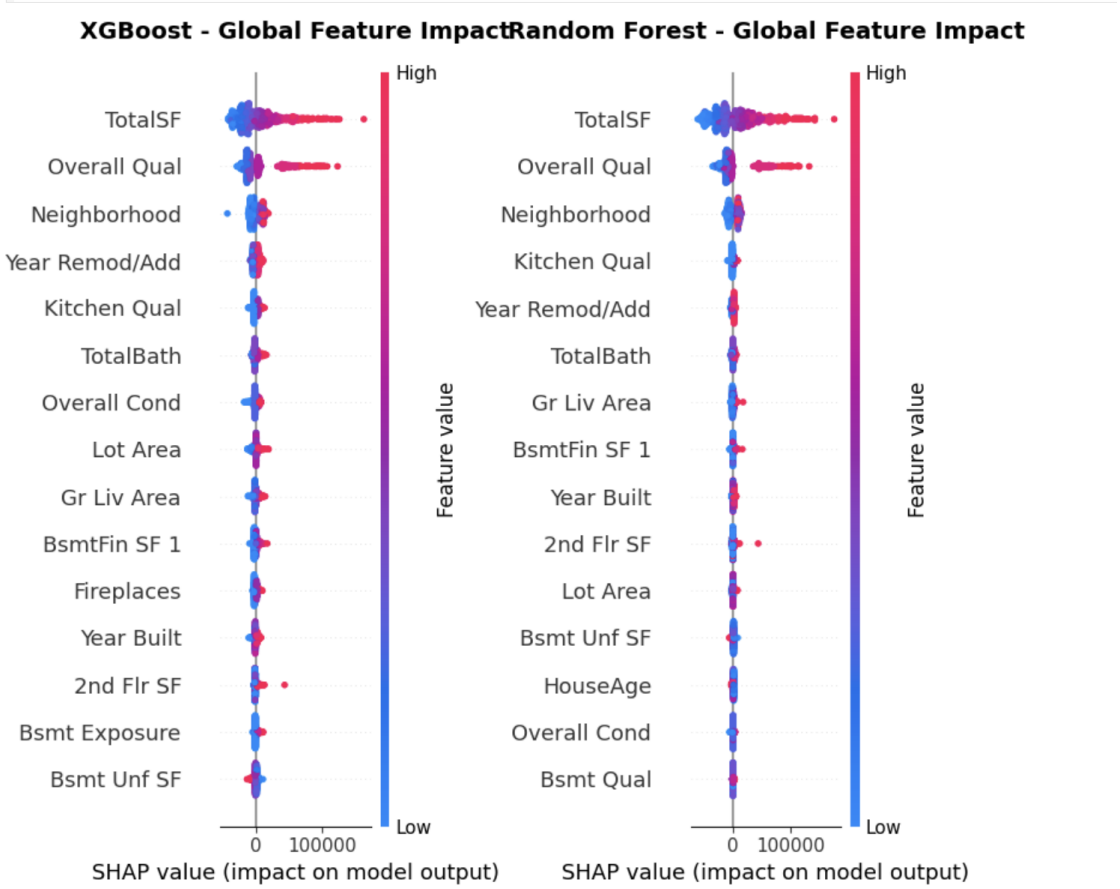

Figure 5: SHAP summary plots for XGBoost and Random Forest on Ames data. Both models identify TotalSF, Overall Qual, and Neighborhood as the most important features.

TotalSF (the engineered feature combining all square footages) dominated with mean SHAP value of $24,543. This validates the feature engineering choice – combining related area measurements into a single total was more useful than keeping them separate.

Overall Qual ranked second, confirming that quality ratings matter a lot for price. Neighborhood ranked third, supporting the location hypothesis.

Interestingly, Gr Liv Area (above-ground living area) ranked only 9th despite being highly correlated with price. This is because TotalSF already captured most of that information, so Gr Liv Area's marginal contribution was small.

4.8 Cross-Regional Transfer Results

Applying the Ames-trained models to King County data without retraining produced dramatically poor results (Table 5).

Table 5: Cross-Regional Transfer Performance

Scenario	RMSE ($)	R^2
XGBoost on Ames test (baseline)	22,108	0.931
XGBoost transfer to King County	492,187	-0.797
Random Forest transfer to King County	510,548	-0.934
Linear Regression transfer to King County	170,828,500	-216,523.68
XGBoost trained natively on King County	230,890	0.647

All three models failed to generalize. XGBoost's RMSE jumped from $22k to $492k, and R^2 became negative (-0.797), meaning the model performed worse than simply predicting the mean price for every Seattle house.

Linear Regression completely exploded with RMSE of $170 million and R^2 of -216,523. This happened because Linear Regression extrapolated wildly when faced with feature values far outside its training distribution.

Interestingly, XGBoost transferred least badly among the three models. Tree-based models handled the distribution shift somewhat better than Linear Regression, though all three were terrible in absolute terms.

Training XGBoost natively on King County data (with its own train-test split from the 21,613 houses) improved RMSE dramatically to $231k with R^2 of 0.647. This is still worse than the Ames performance ($22k RMSE), likely because King County only has 21 features versus Ames's 82, but it's much better than transfer.

4.8.1 Why Transfer Failed

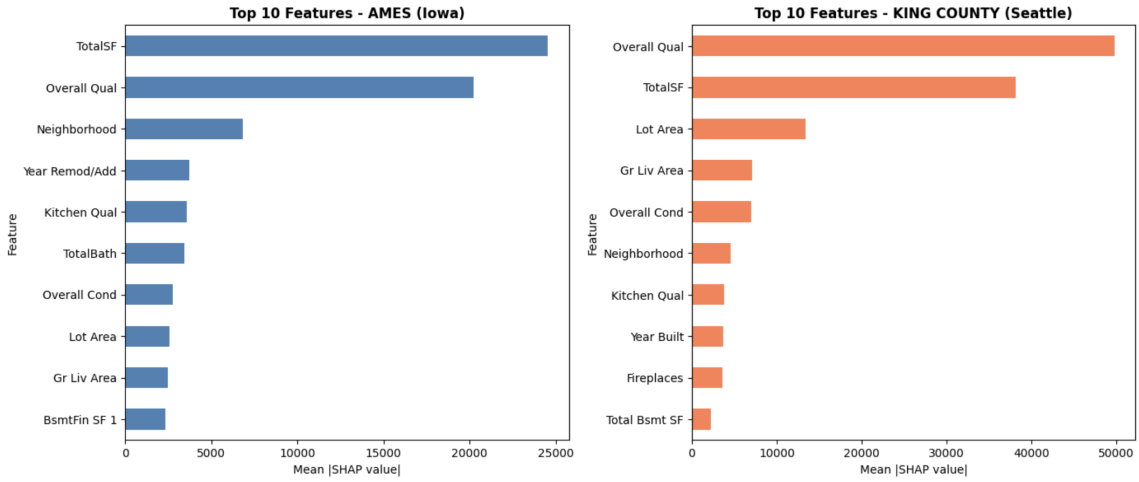

Figure 6: Feature importance comparison between Ames (Iowa) and King County (Seattle). Overall Qual has 2.5x higher importance in Seattle, indicating different market priorities across regions.

Looking at the results, the failure seems to come from a few things.

Feature mismatch: Only 9 of the 174 features could be mapped from King County to Ames format. The remaining 165 features were filled with Ames training medians, creating a very poor representation of Seattle houses.

Geographic differences: Ames, Iowa is a midwestern college town with median home price around $160k. King County includes Seattle, a tech hub with median home price around $450k. The markets have fundamentally different dynamics.

Feature importance shift: SHAP comparison revealed that feature importance differs between regions. Seattle buyers prioritize quality/grade 2.5x more than Iowa buyers (mean —SHAP— of 49,774 vs 20,233). This suggests that what matters for pricing in one market doesn't necessarily matter equally in another market.

4.9 Interpretability Analysis

Comparing Linear Regression (fully interpretable) to XGBoost (more accurate but black-box):

Accuracy gap: XGBoost achieved RMSE of $22,108 versus Linear Regression's $31,268 – a difference of $9,160 or 29% better performance.

Interpretability: Linear Regression coefficients directly show how each feature affects price. For example, "one unit increase in Overall Qual increases price by $12,543."

XGBoost doesn't provide this, but SHAP values offer a similar interpretation: "Overall Qual has mean impact of $20,233 on predictions."

Tradeoff decision: For housing prices, I would choose XGBoost with SHAP. The $9k accuracy improvement matters for real estate decisions (it could change whether someone can afford a house or whether a seller gets their asking price). SHAP provides sufficient interpretability for most use cases – you can explain which features drove a prediction up or down.

However, if I needed to satisfy regulatory requirements (like explaining loan decisions) or present to non-technical stakeholders who want simple rules, Linear Regression's full transparency might be worth the accuracy sacrifice.

5 Discussion and Conclusion

5.1 Key Findings

This project yielded several important insights about housing price prediction:

Tree-based models are essential for this problem. Linear models struggled because housing prices have strong nonlinear relationships and feature interactions. A 1,000 sq ft increase matters more for small houses than huge houses, and location interacts with quality in complex ways. Tree models captured these patterns while linear models couldn't.

Feature engineering had measurable impact. TotalSF became the single most important feature despite being engineered from existing features. This shows that domain knowledge (combining related measurements) can improve model performance.

Models fail predictably on edge cases. Error analysis revealed that luxury homes, rare neighborhoods, and unusual feature combinations all increased prediction error. This isn't random – it's because the model had fewer training examples for these cases and they had higher inherent variance.

Cross-regional generalization is extremely difficult. The Ames model's complete failure on King County (negative R^2) demonstrates that geographic distribution shift is a real problem. You can't just train a model on one city and expect it to work on another without adaptation.

Feature importance varies by market. Seattle buyers weight quality/grade much more heavily than Iowa buyers. This suggests that pricing dynamics differ across markets and a single feature importance ranking doesn't apply everywhere.

SHAP provides practical interpretability. While XGBoost is technically a black box, SHAP analysis made its predictions interpretable enough for practical use. This chal-

lenges the assumption that you must sacrifice accuracy for interpretability.

5.2 Research Questions Answered

Q1: Which model performs best? XGBoost with hyperparameter tuning achieved the lowest RMSE ($22,108) and highest R^2 (0.931). However, Random Forest performed nearly as well and was more stable without extensive tuning.

Q2: Which features are most important? TotalSF, Overall Qual, and Neighborhood were the top three. This matches domain knowledge – size, quality, and location are the classic real estate drivers. The surprise was that engineered TotalSF outperformed individual area measurements.

Q3: Where do models make larger errors? Luxury homes (¿ $300k) had 4x higher absolute error than low-tier homes. Neighborhoods with high internal price variance (like OldTown) also showed elevated error. Very old houses and very large lots were systematically mispredicted.

Q4: Can Iowa models predict Seattle prices? No. Transfer learning failed with negative R^2, meaning the model performed worse than predicting the mean. Feature mismatch and market differences made direct transfer impossible.

Q5: How does feature importance shift across regions? Seattle prioritized quality/grade 2.5x more than Iowa (SHAP value $49,774 vs $20,233). This reflects different market dynamics – Seattle's tech-driven market may value high-end construction more than Iowa's market.

5.3 Implications

For practitioners: Don't assume models trained on one dataset will work on another, even if both are housing prices. Always validate on data from the target distribution. Consider domain adaptation techniques or retraining when entering new markets.

For model selection: Choose models based on use case requirements. If you need regulatory compliance or simple explanations, accept Linear Regression's accuracy limitations. If accuracy is critical and SHAP-level interpretability suffices, use XGBoost.

For feature engineering: Domain knowledge helps. Simple transformations (like combining related measurements into TotalSF) can create more useful features than raw data provides.

5.4 Limitations

This study has several limitations:

Feature mismatch in transfer test. The King County dataset only had 21 features versus Ames's 82, making the transfer test unrealistically difficult. A fairer test would use two datasets with similar feature coverage.

Time period differences. Ames data is from 2006-2010 while King County is from 2014-2015. Housing market dynamics may have changed between these periods, adding another source of distribution shift beyond geography.

Limited transfer learning techniques. I tested naive transfer (use model as-is) but didn't explore domain adaptation methods like reweighting samples, transfer learning with fine-tuning, or learning domain-invariant features. These techniques might improve cross-regional performance.

Single cross-regional pair. I only tested Ames → King County. Testing multiple region pairs (e.g., California → Florida) would show whether the transfer failure generalizes or is specific to Iowa vs Seattle.

5.5 Future Work

Several directions could extend this research:

Test additional cross-regional pairs. Try California Housing → Florida, or Midwest → East Coast, to see if some region pairs transfer better than others.

Implement domain adaptation. Use techniques like importance reweighting, adversarial domain adaptation, or transfer learning with fine-tuning to improve cross-regional performance.

Explore ensemble methods. Combine predictions from models trained on multiple regions, possibly with region-specific weighting.

Deploy as web application. Build a Flask or Streamlit app where users can input house features and get price predictions, making the work more accessible.

Incorporate external data. Add economic indicators (local unemployment, median income), school ratings, crime statistics, or other neighborhood-level features that might reduce error in high-variance neighborhoods.

5.6 Conclusion

This project showed me that housing price prediction is not just about getting high R^2 values. Understanding where models fail, why they fail, and whether they generalize to new

contexts is crucial for real-world deployment.

Tree-based models (particularly XGBoost) substantially outperformed linear models on the Ames dataset, achieving RMSE of $22,108 and R^2 of 0.931. However, this excellent performance did not transfer to King County Seattle data, where the same model produced negative R^2 due to feature mismatch and market differences. This reinforces an important lesson: models don't magically generalize – they're specific to the distribution they trained on.

The error analysis revealed predictable failure modes: luxury homes, rare neighborhoods, and unusual feature combinations all increased prediction error. This suggests that model improvements should focus on these edge cases through better feature engineering, targeted data collection, or specialized sub-models.

SHAP analysis showed that modern interpretability tools allow us to use accurate black-box models without completely sacrificing explainability. While Linear Regression offers perfect transparency, XGBoost with SHAP provides a practical middle ground – high accuracy with adequate interpretability for most use cases.

Overall, this project illustrates the importance of thorough model evaluation beyond test set metrics. Real-world ML applications require understanding model limitations, failure modes, and generalization boundaries. Housing price prediction is not a solved problem – it's a domain where data distribution, geographic context, and market dynamics all matter, and where one-size-fits-all models don't exist.

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Anne C Case and Lawrence F Katz. The company you keep: The effects of family and neighborhood on disadvantaged youths. *NBER Working Paper*, (w3705), 1991.

[3] Category Encoders Contributors. Category encoders: A collection of sklearn transformers to encode categorical variables as numeric. https://contrib.scikit-learn.org/category$_e ncoders/, 2024. Accessed : 2026 - 03 - 24.$

[4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Dean De Cock. Ames, iowa: Alternative to the boston housing data as an end of semester regression project. *Journal of Statistics Education*, 19(3), 2011.

[6] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, pages 1189–1232, 2001.

[7] Google. Google colaboratory. https://colab.research.google.com/, 2024. Accessed: 2026-03-24.

[8] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[9] Arthur E Hoerl and Robert W Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[10] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[11] Kaggle. House sales in king county, usa. https://www.kaggle.com/datasets/harlfoxem/housesalespredic 2015. Accessed: 2026-03-24.

[12] Kaggle. Ames housing dataset. https://www.kaggle.com/datasets/prevek18/ames-housing-dataset, 2016. Accessed: 2026-03-24.

[13] Ron Kohavi et al. A study of cross-validation and bootstrap for accuracy estimation and model selection. 14(2):1137–1145, 1995.

[14] Andy Liaw and Matthew Wiener. Classification and regression by randomforest. *R News*, 2(3):18–22, 2002.

[15] Scott Lundberg. Shap (shapley additive explanations). https://shap.readthedocs.io/, 2024. Accessed: 2026-03-24.

[16] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

[17] Wes McKinney et al. Data structures for statistical computing in python. 445:51–56, 2010.

[18] Christoph Molnar. Interpretable machine learning: A guide for making black box models explainable. 2020.

[19] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2009.

[20] pandas Development Team. pandas: powerful python data analysis toolkit. https://pandas.pydata.org/, 2024. Accessed: 2026-03-24.

[21] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[22] Project Jupyter. Project jupyter. https://jupyter.org/, 2024. Accessed: 2026-03-24.

[23] Python Software Foundation. Python programming language. https://www.python.org/, 2024. Accessed: 2026-03-24.

[24] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D Lawrence. Dataset shift in machine learning. *The MIT Press*, 2009.

[25] Scikit-learn Developers. Scikit-learn: Machine learning in python. https://scikit-learn.org/stable/, 2024. Accessed: 2026-03-24.

[26] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. volume 8, pages 1–21. BioMed Central, 2007.

[27] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.

[28] Michael Waskom. seaborn: statistical data visualization. https://seaborn.pydata.org/, 2024. Accessed: 2026-03-24.

[29] XGBoost Developers. Xgboost documentation. https://xgboost.readthedocs.io/, 2024. Accessed: 2026-03-24.

[30] Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):301–320, 2005.

A AI Tool Usage Disclosure

This project used the following AI tools:

A.1 Chatgpt and Claude AI (Anthropic)

Brainstorming and Planning: Discussed project structure and phase organization. Explored different approaches to error analysis. Generated ideas for cross-regional generalization test.

A.2 Grammarly

Grammar and Proofreading: Checked spelling, grammar, and punctuation throughout the report. Suggested improvements to sentence structure and readability.

All analysis, modeling decisions, interpretations, and conclusions are my own original work. AI tools were used only for technical assistance and writing improvement. AI was not used to write passages, generate code from scratch, or make analytical decisions.